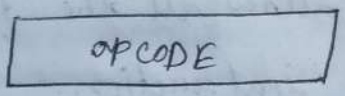
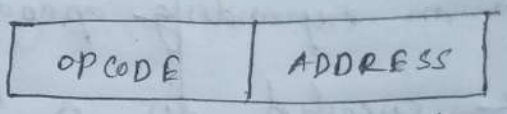
# Lecture 1 - MIPS computer Architecture overview

Book - Computer Organization and Design (Patterson/Hennessy) 5th Edition

① Instruction formats -

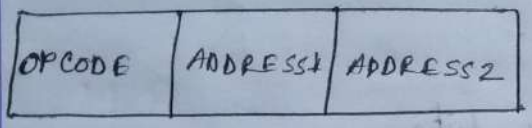



(a) zero address instruction

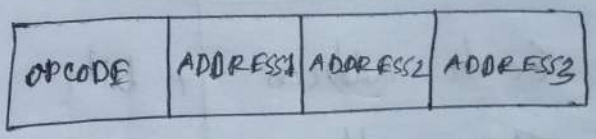



(b) One-address instruction

Book Page 64 (Fig 2.1)




(c) Two-address instruction




(d) Three-address instruction

66 (2.3) Design an expanding opcode to allow all the following to be encoded in a 16-bit instruction:

1. 15 instructions with three 4-bit addresses:

Each address = 4 bits

Three addresses:

$3 \times 4 = 12$ bits

So, 12 bits are used for addresses.

Remaining 4 bits for OPCODE:

$16 - 12 = 4$ bits

4 bit opcode can represent:

$2^4 = 16$ instructions.

We only need 15 instructions.

So,

4 bit op code → 16 combinations

we use 15 of them.

| OPCODE (4) | ADDRESS1 (4) | ADDRESS2 (4) |
|---|---|---|

2. 14 instructions with two 4-bit addresses:

Two addresses:

$2 \times 4 = 8$ bits

Remaing bits for op code:

$16 - 8 = 8$ bits

| OPCODE (8) | ADDRESS 1 (4) | ADDRESS 2 (4) |
|---|---|---|

8 bit op code can represent:

$2^8 = 256$ instructions

we only need 14 instructions

3. 31 instructions with one - 4 bit address:

One address:

$1 \times 4 = 4$ bits

Remaing bits:

~~16 ×~~ $16 - 4 = 12$ bits

12-bit op code gives:

$2^{12} = 4096$ instructions

We only need 31 instructions.

| OPCODE (12) | ADDRESS (4) |
|---|---|

4.16 instructions with no address:

There is no address.
The whole instruction is opcode

| OP CODE |
|---|

MIPS Instruction formats [Book Page 80, 84, 85 (82, 83) must]

| field | 0 | rs | rt | rd | shamt | funct |
|---|---|---|---|---|---|---|
| Bit positions: | 31:26 | 25:21 | 20:16 | 15:11 | 10:6 | 5:0 |

(a) R - type instruction

| field | 35 or 43 | rs | rt | address |
|---|---|---|---|---|
| Bit positions: | 31:26 | 25:21 | 20:16 | 15:0 |

(d) Load or store instruction

| field | 4 | rs | rt | address |
|---|---|---|---|---|
| Bit positions: | 31:26 | 25:21 | 20:16 | 15:0 |

rs - Source Register
rt - Target Register (second source)
rd - Destination Register

Shamt → Shift Amount

funct → function code

OP: Basic operation of the instruction, traditionally called the op code (the field that denotes the operation and format of an instruction).

rs: The first register source operand

add $t0, $t1, $t2

Here, rs = $t1 , rt = $t2 , rd = $t0

rt: The second register source operand

rd: The register destination operand. It gets the result of the operation.

shamt: Shift amount

SLL $t0, $t1, 3

This means:

$t0 = $t1 << 3 ; shamt = 3

It tell how many (3) bit positions to shift.

If instruction is ~~not a~~ NOT a shift instruction → shamt = 0

funct: function. This field, often called the function code, selects the specific variant of the operation in the op field.

In R-type instruction:

# opcode = 000000

# funct field tells which operation to perform.

Ex:

| Instruction | funct value |
|---|---|
| add | 100000 |
| sub | 100010 |
| and | 100100 |
| or | 100101 |

So, the CPU checks:

opcode = 000000 ?

yes ⇒ look at the funct field

Why Both opcode and funct?

Because MIPS keeps R-type instructions grouped under opcode = 0

So instead of using 6 opcode bits for every arithmetic ~~function~~ instruction,

they use :

opcode → tells format type

funct → tells exact operation

## I-Type Instruction (Book page 83)

| op | rs | rt | constant or address |
|---|---|---|---|
| 6 bits | 5 bits | 5 bits | 16 bits |
| 31 : 26 | 25 : 21 | 20 : 16 | 15 : 0 |

The 16 bit address means a load word instruction can load any word within a region of $\pm 2^{15}$ or 32,768 bytes ($\pm 2^{13}$ or 8192 words) of the address in the base register rs. Similarly, add immidiate is limited to constants no longer than $\pm 2^{15}$.

The first three fields of the R-type and I-type formats are the same size and have the same names, the length of the fourth fied in I-type is equal to the sum of the lengths of the last three fields of R-type.

Book Page 64 full copy

MIPs Operand

| Name | Example | Comment |
|---|---|---|
| 32 registers | \$s0 - \$s7, \$t0 - \$t9, \$zero, \$a0 - \$a3, \$v0 - \$v1, \$gp, \$fp, \$sp, \$ra, \$at | Fast location for data. In MIPs, data must be in registers to perform arithmetic, register \$zero always 0, and register \$at is reserved by the assembler to handle large constants. |
| $2^{30}$ memory words | Memory[0], Memory[4], ..... Memory[4294967292] | Accessed only by data transfer instructions. MIPs uses byte addresses, so sequential word addresses differ by 4. Memory holds data structures, arrays and spilled registers |

# MIPS Assembly language

| Category | Instruction | Example | Meaning | Comments |
|---|---|---|---|---|
| Arithmetic | add | add $S1, $S2, $S3 | $S1 = $S2 + $S3 | Three register operands |
| | subtract | sub $S1, $S2 $S3 | $S1 = $S2 − $S3 | Three register operands |
| | add immidiate | addi $S1, $S2, 20 | $S1 = $S2 + 20 | Used to add constants. |
| Data Transfer | load word | lw $S1, 20($S2) | $S1 = Memory [$S2+20] | Word from memory to register |
| | store word | sw $S1, 20($S2) | Memory [$S2+20] = $S1 | Word from register to memory |
| | load half | lh $S1, 20($S2) | $S1 = Memory [$S2+20] | Halfword memory to register |
| | load half unsigned | lhu $S1, 20($S2) | $S1 = Memory [$S2+20] | Halfword memory to register |
| | store half | sh $S1, 20($S2) | Memory [$S2+20] = $S1 | Half word register to memory |
| | load byte | lb $S1, 20($S2) | $S1 = Memory [$S2+20] | Byte from memory to register |
| | load byte unsigned | lbu $S1, 20($S2) | $S1 = Memory [$S2+20] | Byte from memory to register |
| | store byte | sb $S1, 20($S2) | Memory [$S2+20] = $S1 | Byte from register to memory |
| | load linked word | ll $S1, 20($S2) | $S1 = Memory [$S2+20] | Load word as 1st half of atomic swap |
| | store condition word | sc $S1, 20($S2) | Memory [$S2+20] = $S1; $S1 = 0or1 | Store word as 2nd half of atomic swap |
| | load linked word | ll $S1, 20($S2) | $S1 = Memory [$S2+20] | Load word as 1st half of atomic swap |
| | store condition. word | sc $S1, 20($S2) | Memory [$S2+20] = $S1; $S1 = 0or1 | Store word as 2nd half of the atomic swap |
| | load upper immed. | lui $S1, 20 | $S1 = 20 * 2^16 | Loads constans in upper 16 bits. |

# MIPS Assembly language

| Category | Instruction | Example | Meanings | Comment |
|---|---|---|---|---|
| Logical | and | and $S1, $S2, $S3 | $S1 = $S2 & $S3 | Three register operand; bit-by-bit AND |
| | or | or $S1, $S2, $S3 | $S1 = $S2 \| $S3 | Three register operand; bit-by-bit OR |
| | NOR | nor $S1, $S2, $S3 | $S1 = ~($S2 \| $S3) | Three register operand; bit-by-bit NOR |
| | and immidiate | andi $S1, $S2, 20 | $S1 = $S2 & 20 | Bit-by-bit AND reg with constant |
| | or immidiate | ori $S1, $S2, 20 | $S1 = $S2 \| 20 | Bit-by-bit OR reg with constant |
| | Shift left logical | sll $S1, $S2, 10 | $S1 = $S2 << 10 | shift left by constant |
| | Shift right logical | srl $S1, $S2, 10 | $S1 = $S2 >> 10 | shift right by constant |
| Conditional branch | branch on equal | beq $S1, $S2, 25 | if ($S1 == $S2) go to PC+4+100 | Equal test; PC-relative branch |
| | branch on not equal | bne $S1, $S2, 25 | if ($S1 != $S2) go to PC+4+100 | Not equal test; PC-relative |
| | set on less than | slt $S1, $S2, $S3 | if ($S2 < $S3) $S1 = 1 else $S1 = 0 | Compare less than; for beq, bne |
| | Set on less than unsigned | sltu $S1, $S2, $S3 | if ($S2 < $S3) $S1 = 1 else $S1 = 0 | Compare less than unsigned |
| | set on less than immidiate | slti $S1, $S2, 20 | if ($S2 < 20) $S1 = 1 else $S1 = 0 | Compare less than constant |
| | set less than immidiate unsigned | sltiu $S1, $S2, 20 | if ($S2 < 20) $S1 = 1 else $S1 = 0 | compare less than constant unsigned |

MIPs Assembly Language

| Category | Instruction | Example | Meaning | Comment |
|---|---|---|---|---|
| Unconditional jump | jump | j 2500 | go to 10000 | jump to target address |
| | jump register | jr $ra | go to $ra | for switch, procedure return |
| | jump and link | jal 2500 | $ra = PC+4; go to 10000 | For procedure call |

Book page 67,68 example

MIPs Load Store Instructions (Book page 71 example)

In MIPs words must start at addresses that are multiples of 4. This requirement is called alignment restriction.

Compiling using Load and store

Q. Assume variable h is associated with register $s2 and the base address of the array A is in $s3. What is the MIPs assembly code for the C assignment statement below?

Solution:

Given,

h is stored in register $s2

Base address of array A is in the register $s3.

Statement:

A[12] = h + A[8]

Assuming standard MIPS:

Word size = 4 byte

Array elements are integers (1 word each)

Memory is byte-addressable

Address calculation:

In MIPS

Address of A[i] = Base Address + (i × 4)

Because each integer = 4 bytes

Therefore, A[8] = 8 × 4 = 32

∴ Address of A[8] = \$S3 + 32

And so, A[12] = 12 × 4 = 48

∴ Address of A[12] = \$S3 + 48

Translate the expression,

A[12] = h + A[8]

Meaning:

1. Load A[8]
2. Add it with h
3. Store result into A[12]

MIPs Assembly code:

```
lw $t0, 32($s3)      # Temporary reg $t0 gets A[8]
add $t0, $s2, $t0    # Temporary reg $t0 gets h+A[8]
sw $t0, 48($s3)      # stores h+A[8] back into A[12]
```

MIPs Branch Instruction

Here is a simple loop in C:

```
for (i=0; i<4; i++)
{ //stuff }
```

Below is the loop translated into MIPs assembly:
variable i is assigned to $t0

```
    li $t1, 0
    add $t0, $zero, $zero   # i is initialized to 0; $t0=0
Loop: //stuff
    addi $t0, $t0, 1        # $t0++
    slti $t1, $t0, 4        # if ($t0<4) $t1=1 else $t1=0
    bne $t1, $zero, Loop    # go to Loop if $t1!=0
```

## MIPS Instruction Implement Overview

for every instruction, the first two steps are identical:

① Send the program counter (PC) to the memory that contains the code and fetch the instruction from that memory.

② Read one or two registers, using fields of the instruction to select the registers to read. for the load word instruction, we need to read only one register, but most other instructions require reading two registers.

## Building a Datapath

The operations of arithmetic-logical (R-type) instructions and the memory instructions datapath are quite similar. The key differences are the following:

① The arithmetic-logical instruction use the ALU (Arithmetic Logic Unit). The memory instructions can also use ALU to do the address calculation, although the second input is the sign extended 16-bit offset field from the instruction.

② The value stored into a destination register comes from the ALU (for an R-type instruction) or the memory (for a load).

## ALU control

The MIPS ALU control input defines the 6 following combinations of four control inputs:

Book Page 105 Figure 250

Book page 260

| ALU control lines | function |
|---|---|
| 0000 | AND |
| 0001 | OR |
| 0010 | add |
| 0110 | subtract |
| 0111 | set on less than |
| 1100 | NOR |

How the ALU control bits are set depends on the ALUOp control bits and the different function codes for the R-type instruction.

| Instruction Opcode | ALUOp | Instruction operation | Funct field | Desired ALU Action | ALU control input |
|---|---|---|---|---|---|
| LW | 00 | load word | XXXXXX | add | 0010 |
| SW | 00 | store word | XXXXXX | add | 0010 |
| Branch equal | 01 | branch equal | XXXXXX | subtract | 0110 |
| R-type | 10 | add | 100000 | add | 0010 |
| R-type | 10 | subtract | 100010 | subtract | 0110 |
| R-type | 10 | AND | 100100 | AND | 0000 |
| R-type | 10 | OR | 100101 | OR | 0001 |
| R-type | 10 | set on less than | 101010 | set on less than | 0111 |

Diagram on book page 264

figure on page 265

Datapath for an R-type instruction. (Book page 266)

add $t1, $t2, $t3

1. The instruction is fetched, and the PC is incremented.

2. Two registers, $t2 and $t3, are read from the register file; also, the main control unit computes the setting of the control lines during this step.

3. The ALU operates on the data read from the register file, using the function code (bits 5:0, which is the funct field, of the instruction) to generate the ALU function.

4. The result from ALU is written into the register file using bits 15:11 of the instruction to ~~read~~ select the destination register ($t1).

| field | 0 | rs | rt | rd | shamt | funct |
|---|---|---|---|---|---|---|
| Bit positions | 31:26 | 25:21 | 20:16 | 15:11 | 10:6 | 5:0 |

R-type instruction

Datapath for a Load instruction (Book Page 267)

lw $t1, offset($t2)

| Field | 35 or 43 | rs | rt | address |
|---|---|---|---|---|
| Bit Position | 31:26 | 25:21 | 20:16 | 15:0 |

Load or store instruction

1. An instruction is fetched from the instruction memory, and the PC is incremented.

2. A register ($t2) value is read from the register file.

3. The ALU computes the sum of the value read from the register file and the sign-extended, lower 16 bits of the instruction (offset).

4. The sum from the ALU is used as the address for the data memory.

5. The data from the memory unit is written into the register file; the register destination is given by bits 20:16 of the instruction ($t1).

Datapath for a branch-on-equal instruction.

beq $t1, $t2, offset (Book Page 268, 269)

| field | 4 | rs | rt | address |
|---|---|---|---|---|
| Bit positions | 31:26 | 25:21 | 20:16 | 15:0 |

Branch instruction

1. An instruction is fetched from the instruction memory, and the PC is incremented.

2. Two registers; $t1 and $t2, are read from the register file.

3. The ALU performs a subtract on the data values read from the register file. The value of PC+4 is added to sign-extended, lower 16 bits of the instruction (offset) shifted left by two; the result is the branch target address.

4. The zero result from the ALU is used to decide which adder result to store into the PC.

figure in page 273

MIPs Pipeline ~~(Page 274)~~ (Book Page 286)

MIPs instructions classically take five steps:

1. IF: fetch instruction from memory
2. ID: Read registers while decoding the instruction. The regular format of MIPs instructions allows reading and decoding to occur simultaneously.
3. EX: Execute the operation or calculate an address.
4. MEM: Access an operand in data memory.
5. WB: Write the result into a register.

figure in page 287

figure in page 289

⇓

IF/ID registers must be 64 bits wide, because it must hold both 32-bits instruction fetched from memory and the incremented 32-bit PC address.

Page 264 and 302 have same table but 302 is better.

Page 275, 276

## Pipeline Hazards (Book page 277)

There are situations in piplining when the next instruction cannot execute in the following clock cycle. These events are called hazards, and there are three different types:

1. Structural hazard:

When a planned instruction cannot execute in the proper clock cycle because the hardware does not support the combination of instructions that are set to execute.

For example; If there is no separation between data memory and instruction memory, the IF (Fetch Instruction) and MEM (Write Memory) stages cannot occur simultaneously.

2. Data Hazard: (Book page 278)

Occurs when the ~~pipli~~ pipeline must be stalled because one step must wait for another to complete.

Solution in figure 4.29 (279 page) and 4.30 (Page 280)

Page 280,281 Example

3. Control Hazard (Book page 281)

Arising from the need to make a decision based on the results of one instruction while others are executing.

figure 4:32 in page 283

The End